

SNN Network Simulation

Documentation

Table of contents

1	PROJECT INTRODUCTION	4
1.1	OBJECTIVES	4
1.2	MODELLING OVERVIEW.....	4
1.3	SIMULATION SOFTWARE OVERVIEW.....	4
1.4	DOCUMENT ORGANIZATION.....	5
2	NEUROLOGICAL STUDY	6
2.1	INTRODUCTION	6
2.2	BIOLOGY AND MATHEMATICS.....	6
2.2.1	NEURON.....	6
2.2.2	SYNAPSE.....	8
2.2.3	ASTROCYTE	9
2.2.4	NETWORK	11
2.2.5	BIOLOGICAL BASIS OF EPILEPTIC ACTIVITY	12
2.3	MATHEMATIC AND ALGORITHMS	13
2.3.1	VARIABLES MAPPING	13
2.3.2	ALGORITHM	13
3	SIMULATION SOFTWARE DESIGN.....	14
3.1	PRESENTATION.....	14
3.2	LIBRARIES REPOSITORY	14
3.3	FUNCTIONAL VIEW.....	15
3.3.1	MAIN WINDOW.....	15
3.3.2	CONTROL PANEL	15
3.3.3	NETWORK VIEW.....	16
3.3.4	PLOT PANEL	16
3.4	STATIC VIEW.....	17
3.4.1	PACKAGES INVENTORY.....	17
3.4.2	PACKAGE NETWORK.....	17
3.4.3	PACKAGE DATA.....	18
3.4.4	PACKAGE SIMULATION	18
3.4.5	PACKAGE VIEW	19
3.5	DYNAMIC VIEW.....	20
4	CONCLUSIONS	22
5	APPENDIX	23
5.1	VARIABLES MAPPING	23
5.2	PYTHON LIBRARIES	24
5.3	INSTALLATION GUIDE	25
5.4	USER GUIDE	25
5.5	HODGKIN-HUXLEY VARIABLES	26
5.5.1	SODIUM ACTIVATION GATE (M)	26
5.5.2	SODIUM INACTIVATION GATE (H).....	27
5.5.3	POTASSIUM ACTIVATION GATE (N).....	27

Project purpose

This project has two purposes: the first is to build a plausible neural network biologically within a defined scale for observing the difference of behaviour between normal and epileptics parameters. The second is to embed the calculus part in a simulation software offering: user interactions management, outcomes graphic visualisation and capability to export outcomes in file.

An Installation Guide and a User Guide are provided in this document to support installation and software usage (See appendix: [5.3 Installation guide](#) and [5.4 User guide](#))

1 Project introduction

1.1 Objectives

The objectives of this project include neural network modelling and implementation of a simulation software based on this modelling:

- The modelling part relies on Hodgkin-Huxley model to simulate neurons behaviours. First, a set of neurons is defined, they are connected by synapses and regulated by astrocytes. Once done, the objective is to model neural dynamics and to reproduce normal or pathologic behaviour over the time by fitting the parameters.
- The simulation software initializes its context based on neurons dataset and simulation parameters. Then a « real-time » simulation is launched, which provide a graphic display of simulation's outputs and allow user interactions as change simulation setting and export results in a file. The long-term objective is to realize a design that can be easily reuse on new projects with the ability to integrate other neural models and working at different scales.

1.2 Modelling overview

The modelling part includes four major topics related to:

- Neuron: Define neurons equations, by including ions and membrane dynamics,
- Synapses: Define how the neurons interact together and models the membrane dynamics between pre-synaptic and post-synaptic neuron,
- Astrocytes: Define how astrocytes regulate the synapses depending of their activity,
- Network: Define the network rules and topology.

1.3 Simulation software overview

To evaluate the behaviour of this model, a software program has been developed including two main parts:

- A Human-Machine Interface (HMI) allowing a user to:
 - Define the initial simulation parameters,
 - Modify certain parameters during the simulation,
 - Start or stop the simulation,
 - Display the simulation results in the form of a neural network and also physical quantities over the time (2D graph),
 - Export the simulation data into a file.
- An algorithmic module, entirely controlled by the user interface, which performs the following processing:
 - Launching a simulation at T time using the parameters defined in the HMI section and relying on an existing algorithm « Brian2 »,
 - Launching a simulation at T+1 time using the results of the previous iteration and still relying on the HMI parameters and the « Brian2 » algorithm,
 - From the moment the user launches the simulation via the HMI, this module runs in an infinite loop. The simulation ends when the user decides to stop it via a button in the HMI.

1.4 Document organization

This document describes the SNN Network Simulation project by addressing the following themes:

- Presentation of theoretical aspects of the neurological mechanisms studied, which are the subject of the simulation carried out with the developed simulation software,
- Description of the developed software, presenting the functional view of the user interface (user interaction means, display results capabilities, etc.), the static view showing the organization into packages and classes, and finally the dynamic view describing the sequence of the simulation loop process, which is the central element of the simulation.
- Presentation of the project's conclusions on the topics covered: neurological theoretical mechanisms and the technical implementation of the developed simulation software. This also includes identifying the difficulties encountered and areas of improvement that should be considered for a new simulation project.

2 Neurological study

2.1 Introduction

The first objective of this project is to develop a biologically inspired neuron–astrocyte network model capable of reproducing realistic neural dynamics and interaction mechanisms between neurons, synapses, and astrocytes.

The model is based on the Hodgkin–Huxley formalism in order to preserve a biophysical representation of neuronal activity through ionic membrane dynamics. Synaptic transmission is represented using conductance-based interactions, allowing neurons to dynamically influence one another through the propagation of electrical activity across the network.

In addition to neuronal dynamics, the project integrates astrocytes as active regulatory elements capable of modulating synaptic transmission according to the global activity of the network. This neuron–astrocyte coupling introduces additional feedback mechanisms and allows the study of synchronization phenomena, oscillatory activity, and pathological behaviours such as epileptic-like hyperactivity.

The following sections present the biological foundations and mathematical formalization underlying the different components of the model.

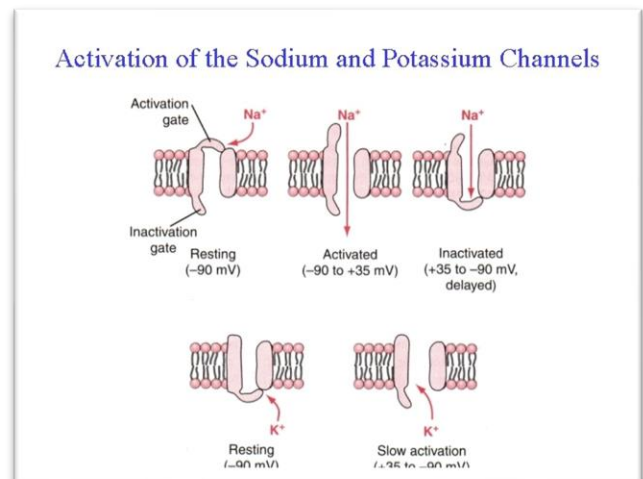
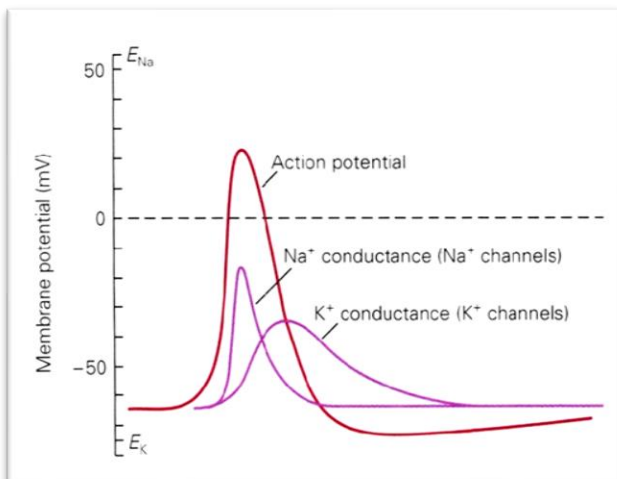
2.2 Biology and Mathematics

2.2.1 Neuron

Neurons communicate through electrical signals generated across their cellular membrane. This electrical activity is produced by differences in ionic concentrations between the intracellular and extracellular environments. The membrane selectively regulates ionic exchanges through specialized membrane proteins known as ion channels.

Among the different ionic species involved in neuronal activity, sodium and potassium ions play a central role in the generation of action potentials. At resting state, the membrane potential remains stable due to the balance between ionic gradients and membrane permeability. When the membrane is sufficiently depolarized, voltage-gated sodium channels rapidly open, allowing sodium ions to enter into the cell. This inward current further amplifies depolarization and generates the rising phase of the action potential.

After this rapid depolarization, potassium channels progressively open and allow potassium ions to leave the cell. This outward current repolarizes the membrane and eventually restores the resting state. The temporal interaction between sodium activation and potassium-mediated repolarization creates the characteristic spike dynamics observed in biological neurons.



The Hodgkin–Huxley formalism mathematically describes these ionic mechanisms using voltage-dependent conductance's and gating variables. This conductance-based representation allows the model to reproduce realistic neuronal excitability and spike generation dynamics. In this framework, the neuronal membrane is represented as an electrical circuit composed of capacitive and conductive elements.

The evolution of membrane potential is governed by the conservation of electrical currents:

$$C_m \frac{dV}{dt} = I_{ext} - I_{Na} - I_K - I_L - I_{syn}$$

C_m : Membrane capacitance,
 I_{ext} : Externally injected current,
 I_{Na} : Sodium current,
 I_K : Potassium current,
 I_L : Leakage current,
 I_{syn} : Synaptic current generated by network interactions.

The ionic currents are modelled through voltage-dependent conductance's controlled by gating variables.

The sodium current is defined by:

$$I_{Na} = g_{Na} m^3 h (V - E_{Na})$$

g_{Na} : The maximal sodium conductance,
 m : Sodium channel activation,
 h : Sodium channel inactivation,
 E_{Na} : The sodium reversal potential.

The potassium current is defined by:

$$I_K = g_K n^4 (V - E_K)$$

g_K : The maximal potassium conductance,
 n : Controls potassium channel activation,
 E_K : The potassium reversal potential.

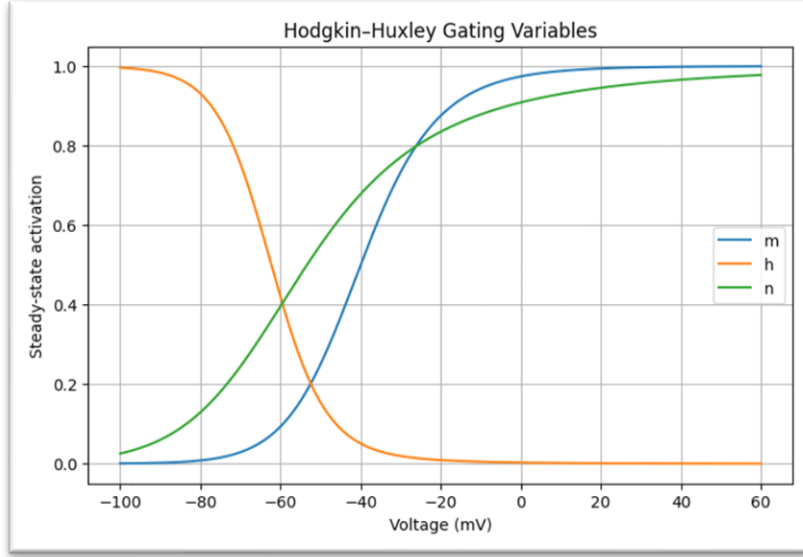
The leaks current is defined by:

$$I_L = g_L (V - E_L)$$

g_L : Maximum leaks conductance,
 E_L : Leaks reversal potential.

SNN Network Simulation - Documentation

The schema hereafter represents the Hodgkin-Huxley Gating variables. The equations of m , h , n will not be described here, to avoid overloading the documentation, there are presented in appendix (See [5.5 Hodgkin-Huxley variables](#)).



The interaction between these ionic currents generates excitable membrane dynamics and allows the spontaneous emergence of action potentials.

The synaptic current will be explained in the next paragraph on synapses.

2.2.2 Synapse

Neural computation does not emerge from isolated neurons but from interactions between interconnected neural populations. These interactions occur through synapses, which transmit information from a pre-synaptic neuron to a post-synaptic neuron.

When a pre-synaptic neuron generates an action potential, neurotransmitters are released into the synaptic cleft. These neurotransmitters modify the membrane permeability of the post-synaptic neuron by opening specific ion channels. As a consequence, synaptic activity generates additional membrane currents capable of influencing post-synaptic excitability.

In the present model, synaptic transmission is represented using a conductance-based formalism. Rather than directly injecting current into the post-synaptic neuron, synaptic activity dynamically modifies membrane conductance. This approach preserves the biological interpretation of synaptic transmission and allows synaptic effects to depend on membrane potential.

Synaptic conductance evolve dynamically over time. Following a pre-synaptic spike, synaptic conductance rapidly increases and then progressively decays. This temporal evolution introduces transient interactions between neurons and allows the emergence of complex collective behaviours at the network level.

When many neurons interact through recurrent synaptic connections, the network can generate synchronized oscillations, bursting activity, propagating waves, and dynamical patterns.

Neurons interact through conductance-based synaptic currents. For a synaptic connection between neuron i and neuron j , the synaptic current is defined by:

$$I_{syn}^{(i,j)} = g_{ij}(E_{syn} - V_j)$$

g_{ij} : Synaptic conductance,

E_{syn} : Synaptic reversal potential,

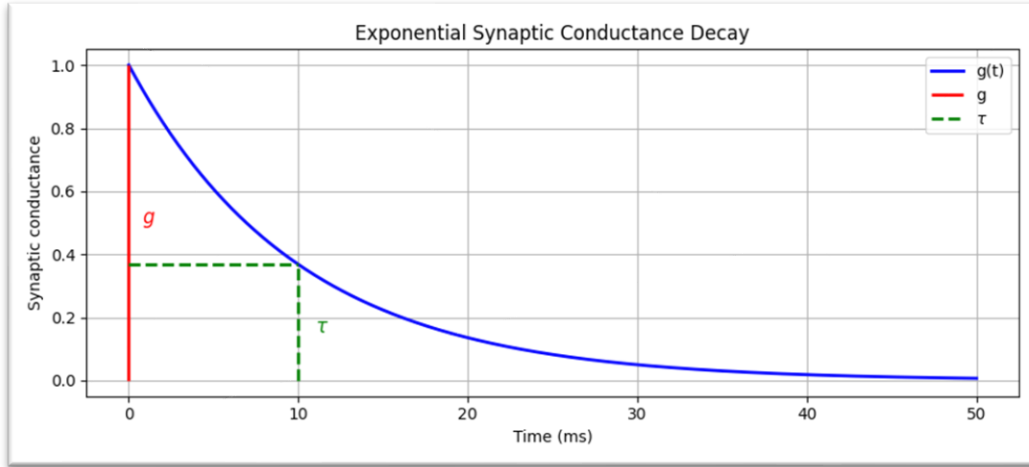
V_j : The post-synaptic membrane potential.

SNN Network Simulation - Documentation

Synaptic conductance evolves dynamically according to:

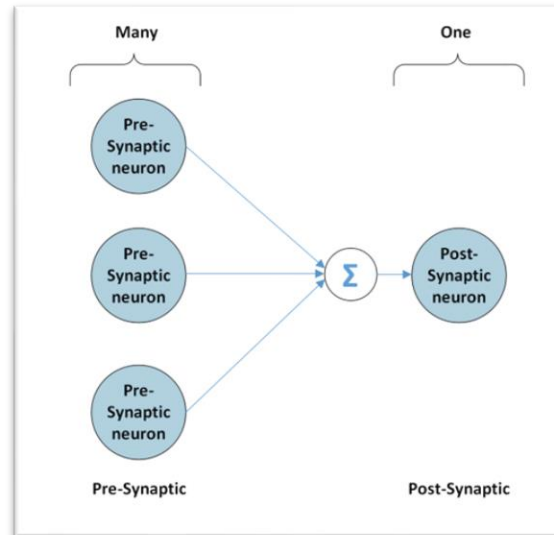
$$\frac{dg_{ij}}{dt} = -\frac{g_{ij}}{\tau_{ij}}$$

This equation models the progressive decay of synaptic influence over time following pre-synaptic activation.



The use of dynamic conductance's allows the network to reproduce transient interactions and temporally structured neural communication.

The total synaptic current $I_{syn,j}$ is obtained by summing all individual synaptic contributions from pre-synaptic neurons. In the implementation, this summation is performed explicitly within a network operation, which iterates over all synapses and accumulates their contributions for each post-synaptic neuron.



2.2.3 Astrocyte

Astrocytes are glial cells traditionally associated with metabolic and structural support functions in the central nervous system. Astrocytes interact closely with synapses and contribute to the regulation of the extracellular neural environment. Their functions include potassium buffering, neurotransmitter uptake,

calcium signalling, and gliotransmitter release. Through these mechanisms, astrocytes are capable of modulating neuronal communication and influencing network excitability.

One important characteristic of astrocyte dynamics is their slower temporal scale compared to neuronal activity. While neurons generate action potentials on the millisecond scale, astrocyte activity evolves more progressively through slower intracellular calcium dynamics and long-term regulation processes.

In the present model, astrocytes are represented as dynamical regulatory units integrating synaptic activity over time. Instead of directly generating spikes, astrocytes accumulate information about network activity and progressively modulate synaptic transmission parameters.

This creates a bidirectional interaction between neurons and astrocytes: neuronal activity influences astrocyte activation, astrocytes modify synaptic transmission, modified synapses alter future neuronal activity.

Such feedback loops introduce additional dynamics into the network and significantly increase the complexity of collective behaviour.

Astrocyte activity is modelled as a slow dynamical variable integrating synaptic activity across the network.

The astrocyte dynamics are defined by:

$$\frac{dA_k}{dt} = -\frac{A_k}{\tau_A} + k \sum_j g_{ij}$$

- A_k : Astrocyte activity,
- τ_A : The astrocyte temporal scale,
- $\sum$: The integrated synaptic activity received by the astrocyte.

This formulation models astrocytes as slow integrators of network activity. Unlike neurons, astrocytes do not react instantaneously but progressively accumulate information about the collective state of the network.

Astrocyte activity dynamically modifies synaptic transmission properties.

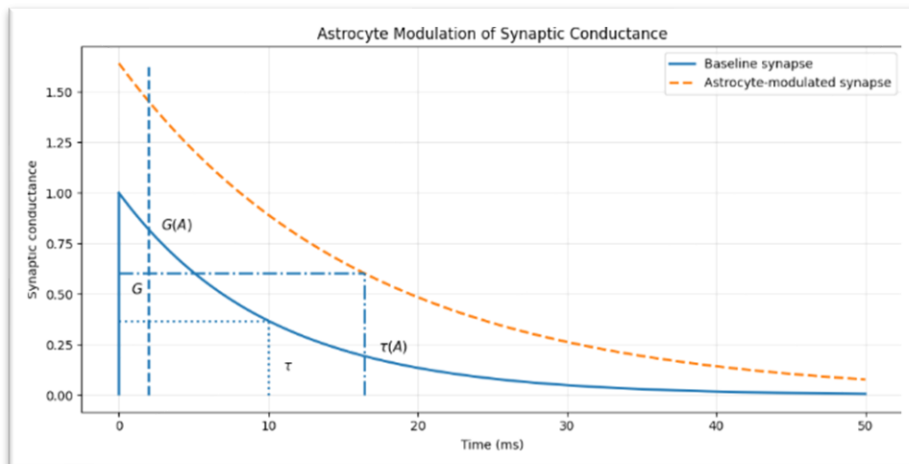
The synaptic weight is modulated according to:

$$w_{ij}(A_k) = w_0(1 + \alpha A_k)$$

While the synaptic decay constant evolves according to:

$$\tau_{ij} = \tau_0 + \beta A_k$$

These equations describe how astrocyte activity modifies both synaptic strength and temporal persistence.



As astrocyte activity increases, synaptic interactions can become stronger and longer-lasting, thereby modifying the propagation of neural activity through the network.

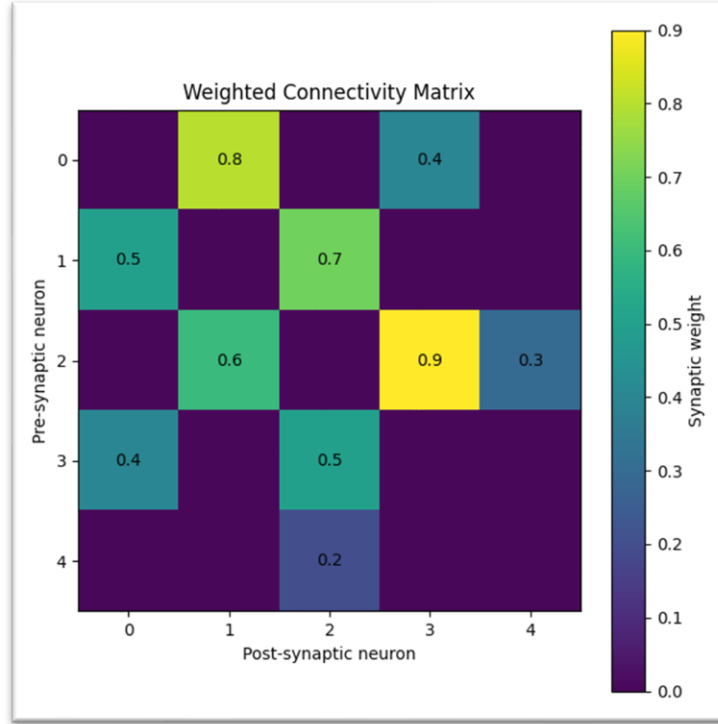
2.2.4 Network

The organization of the neural network is represented through a connectivity matrix:

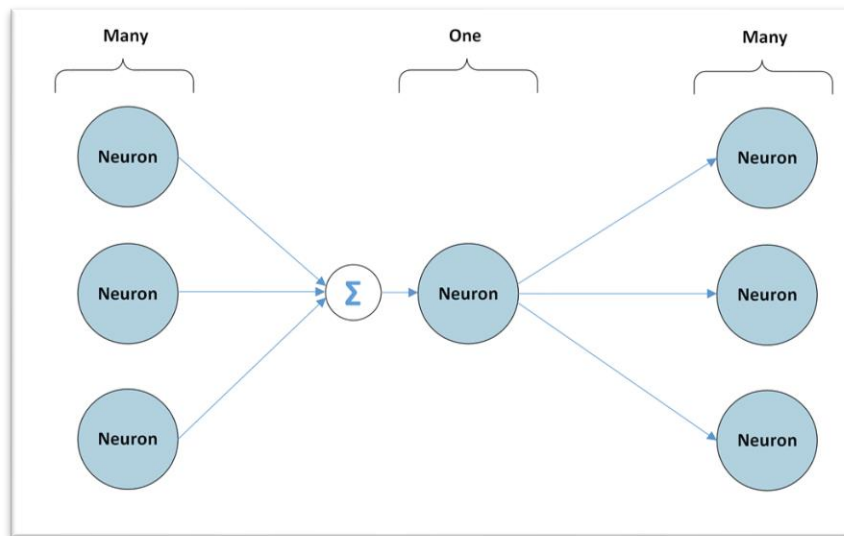
$$W \in \mathbb{R}^{N \times N}$$

Each coefficient W_{ij} represents the existence and strength of a synaptic connection between neuron i and neuron j .

A nonzero value indicates the presence of a synaptic connection, whereas a zero value corresponds to the absence of interaction.



This graph-based representation allows the model to describe neural networks and study how connectivity topology influences global dynamical behaviour.



The interaction between neuronal excitability, synaptic connectivity, and astrocyte-mediated modulation generates a highly dynamical system.

The astrocyte model works on the same ideas than neurons. The distance shown in the network view is « artificial » and added after, and it's only a graphical distance which have no meaning biologically, and no mathematical impact.

The addition of astrocyte-mediated feedback significantly enriches the temporal structure of the network by introducing slower regulatory processes interacting with fast neuronal dynamics.

This coupled architecture therefore provides a framework for studying how local neuron–astrocyte interactions influence collective neural behaviour.

2.2.5 Biological basis of Epileptic Activity

Epilepsy is a neurological disorder characterized by abnormal neuronal hyperexcitability and hyper synchronization. During epileptic seizures, large populations of neurons exhibit excessive and coordinated electrical activity that disrupts normal brain function.

At the biological level, several mechanisms may contribute to seizure generation and propagation. These mechanisms include excessive excitatory neurotransmission, reduced inhibitory regulation, recurrent excitation, abnormal neuronal synchronization and extracellular potassium accumulation. Together, these processes destabilize normal network dynamics and facilitate pathological activity propagation across neuronal populations.

Astrocytes also play a major role in the regulation of neuronal excitability. Under physiological conditions, astrocytes contribute to neural homeostasis through extracellular potassium buffering, glutamate uptake, metabolic support and modulation of synaptic activity. Dysfunction of these regulatory mechanisms can increase extracellular excitability, impair ionic balance, and promote pathological synchronization associated with epileptic activity.

For this reason, neuron–astrocyte interaction models provide an interesting computational framework for studying transitions between physiological and pathological network states. By modifying neuronal, synaptic, and astrocytic parameters, the model can reproduce alterations in network stability, synchronization, and excitability associated with epileptic-like dynamics.

This table presents the values which change between normal and epileptic mode:

Parameter	Normal Value	Epileptic Value	% Change vs Normal	Biological Interpretation
gK	36 mS/cm ²	27 mS/cm ²	-25.0%	Reduced potassium conductance → weaker repolarization and higher excitability
gL	0.30 mS/cm ²	0.20 mS/cm ²	-33.3%	Reduced leak conductance increases membrane resistance and excitability
EI	-54 mV	-51 mV	+5.6%	Leak reversal potential becomes less negative → depolarization tendency
I_bg	0.5	0.75	+50.0%	Higher tonic input current increases spontaneous firing
I_noise	0.2	0.10	-50.0%	Reduced noise may produce more synchronized firing
tau_gsyn	5 ms	7.5 ms	+50.0%	Longer synaptic conductance decay prolongs excitation
w0_exc	0.10 mS/cm ²	0.15 mS/cm ²	+50.0%	Stronger excitatory synapses
w0_inh	0.15 mS/cm ²	0.09 mS/cm ²	-40.0%	Weaker inhibitory synapses
tau_A	100 ms	70 ms	-30.0%	Faster astrocytic modulation dynamics
alpha	0.20	0.30	+50.0%	Stronger astrocyte-to-synapse modulation
connection_prob	0.35	0.50	+42.9%	Denser functional connectivity

2.3 Mathematic and algorithms

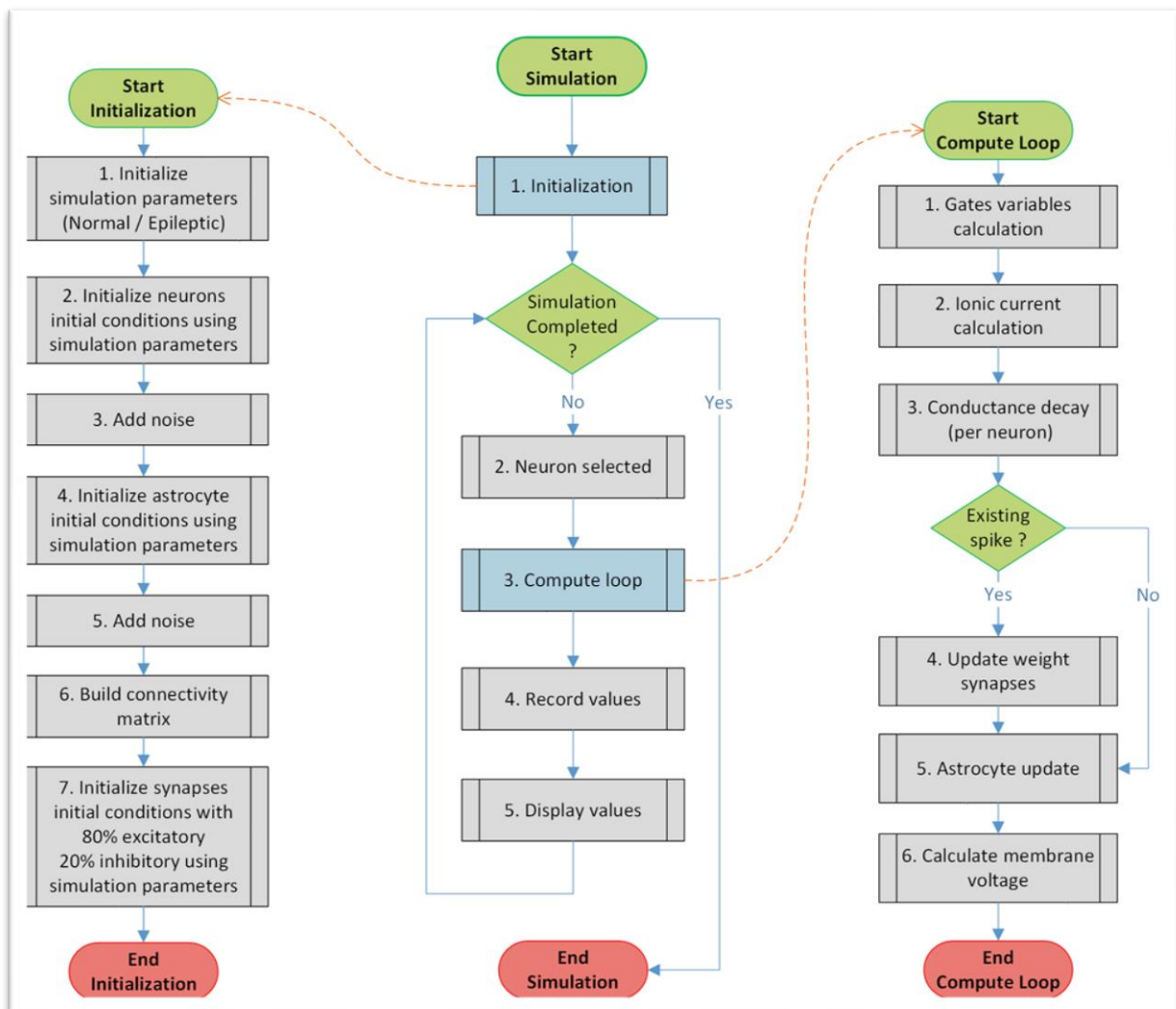
2.3.1 Variables mapping

As programming languages cannot handle Greek alphabet's letters, indices and various mathematic notations, it is necessary to define a mapping between mathematical variables used in biology topics and programmatic variables used to implement algorithms in a programming language (Here Python language).

A mapping table is provided in the appendix that could be a support to enter deeper in the understanding of the simulation algorithm implementation (See [5.1 Variables mapping](#)).

2.3.2 Algorithm

The following schema provides an overall synthesis of the algorithms put in place:



All biology and mathematics topics described in previously are implemented in the « Compute Loop » sequence (In the right part of the sequence diagram). This permit to isolate the treatments performed by the model from the rest of the overall algorithm and thus potentially facilitate the integration of other models' implementation in the future.

The simulations performed were based on a dataset of 15 neurons and 3 astrocytes (These values can be configured)

3 Simulation software design

3.1 Presentation

The simulation software is written in Python and consists of two main parts:

- A user interface that allows for:
 - Gather user events and trigger the requested treatments,
 - Display graphical data (e.g., 2D graphs) to present simulation results to the user.
- An « algorithmic » component:
 - It allows to simulate the behaviour of a neural network over the time and taking into account user-specified input parameters,
 - This component is based on the pre-existing model: Brian2.

3.2 Libraries repository

The software uses the following main external Python libraries to perform its processing:

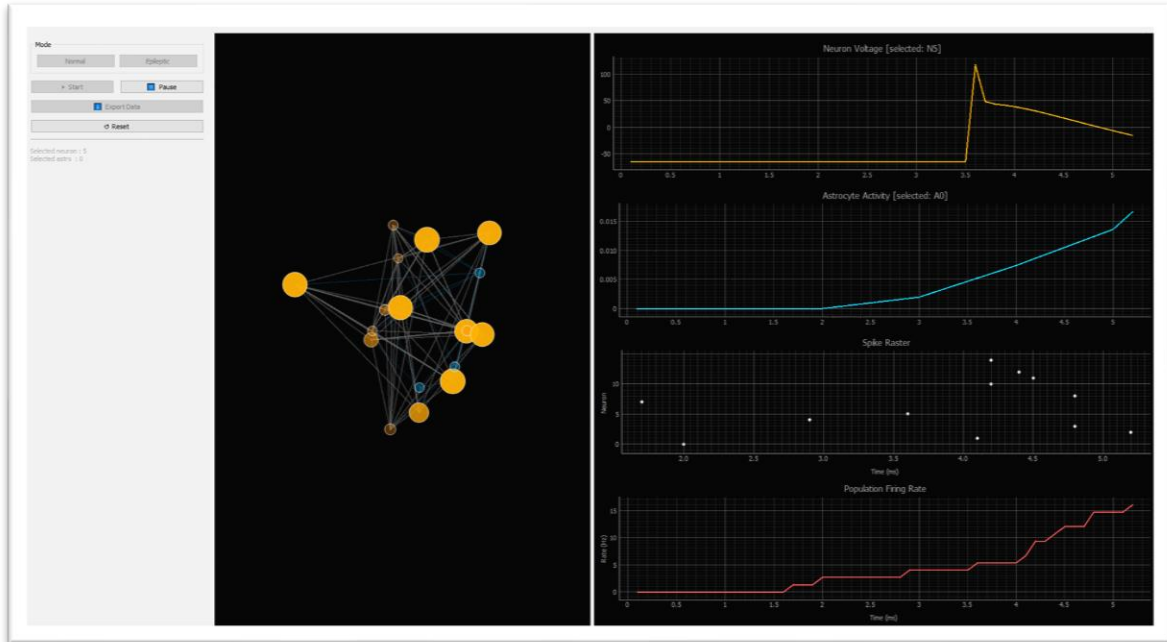
- Brian2: Simulator for spike neural network
- Pyqtgraph: Python graphic and GUI (Graphic User Interface) library built on PyQt/numpy
- PyQt5: Qt is a software framework and a set of tools for creating graphical user interfaces and applications that run on various platforms, PyQt is the Python version of Qt.
- Numpy: The fundamental package for N-dimensional arrays, mathematical functions, and numerical computing in Python

Other « standard » libraries are required to execute the software; a complete list is available in appendix (See [5.2 Python libraries](#)).

3.3 Functional view

3.3.1 Main Window

The application screen that appears when the software is launched is as follows:

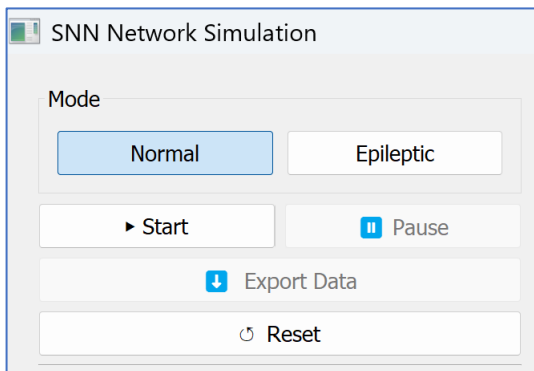


This window is divided into three vertical zones:

- The **control panel** (located on the left):
It contains the buttons that the user can use to trigger actions.
- The **network view** (located in the centre):
It contains the representation of the neurons network being simulated.
- The **plot panel** (located on the right):
It contains the 2D representations of the variables: voltage, astrocyte, raster, and population firing rate over the time during the simulation.

3.3.2 Control Panel

The control panel looks like this:



It contains the following buttons:

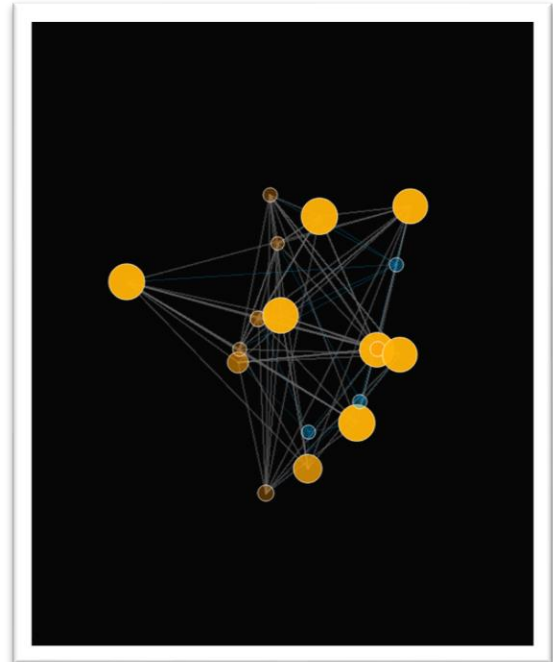
- **Normal** and **Epileptic** modes permit to run the simulation with or without considering the epilepsy context.
- **Start** and **Pause** buttons allow to start or pause the simulation, respectively.
- **Export Data** permits to export the simulation data into a file, once the simulation is completed.
- **Reset** performs a reset of the simulation parameters.

3.3.3 Network View

The network view is a graphical area that represents the state of the neurons network. The two representations below show the appearance of the network in the case of a simulation that has not yet started and the case where the simulation is running:



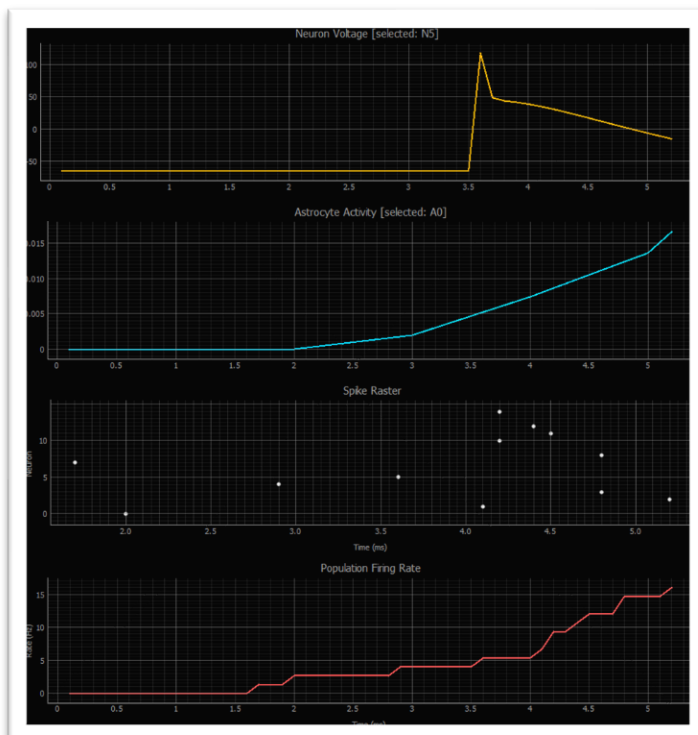
Simulation not started.



Simulation in progress.

3.3.4 Plot Panel

The plot panel area is presented as follows



The plotted variables are:

Variable	Unit
Neuron Voltage	mV
Astrocyte Activity	N/A
Spike Raster	Neurons
Population Firing Rate	Hertz

The **Neuron Voltage** variable represents the voltage of a previously selected neuron.

Similarly, the **Astrocyte Activity** represents the activity of a previously selected astrocyte.

3.4 Static view

The static view shows the software's organization into packages and classes. A package is a group of classes that work together to perform homogeneous treatments.

3.4.1 Packages inventory

The package decomposition that was selected for the simulation software is as follows:

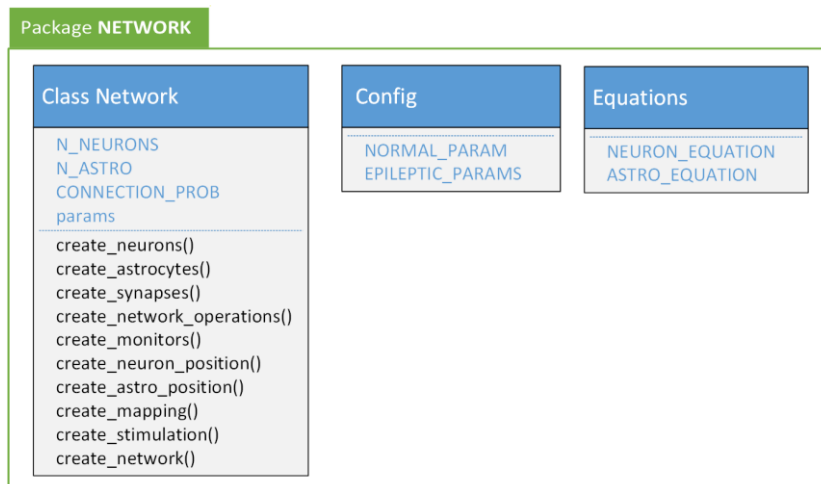
Package	Description
DATA	This package manages and stores the data from the simulation.
NETWORK	This package manages the creation and contains the treatments related to the simulated neural network.
SIMULATION	This package implements the simulation algorithms at high level. It establishes the link between the algorithmic logic and the HMI actions (User interaction & Data visualisation).
VIEW	This package contains the treatments related to the user interface; it includes the creation of graphical objects, their updates, and the treatments to be triggered following user actions.

3.4.2 Package NETWORK

This package includes the following classes:

Class	Description
Network	This class define the composition of a neural network in terms of characteristics: neurons, synapses, and astrocytes. It also contains treatment to support algorithms associated to neural network.
Config	This module defines and centralizes the constants used to implement the simulation parameters for normal and epileptic mode (Note: It is a Python module, not a class).
Equations	This class centralizes the values of the various equations as NEURON_EQUATION, ASTRO_EQUATION, SYNAPSE_EQUATION, etc... which are used in the invocation of the Brian2 library. (Note: It is a Python module, not a class).

The following diagram illustrates the NETWORK package with its classes (attributes and methods):

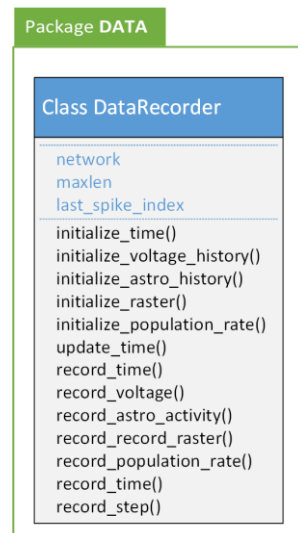


3.4.3 Package DATA

This package includes the following classes:

Class	Description
DataRecorder	This class permits to store simulation data for each iteration. The data recorded includes: voltage, astrocyte, raster, and population firing rate.

The following diagram illustrates the DATA package with its classes (attributes and methods):

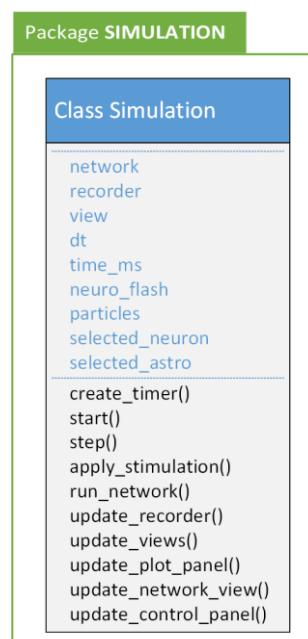


3.4.4 Package SIMULATION

This package includes the following classes:

Class	Description
Simulation	This class is a kind of orchestrator making the link between the algorithmic logic and the HMI, ensuring taking into account user interactions and the simulation data visualisation.

The following diagram illustrates the SIMULATION package with its classes (attributes and methods):

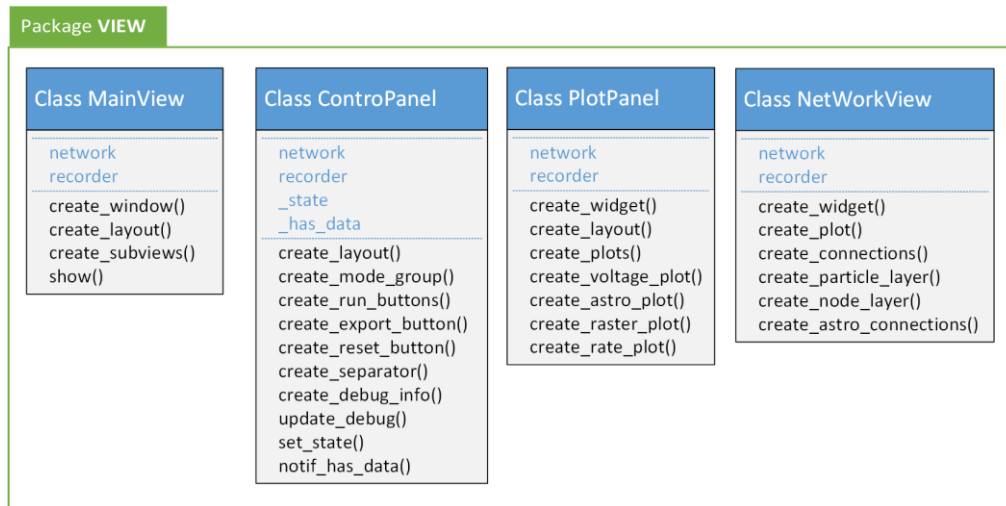


3.4.5 Package VIEW

This package includes the following classes:

Class	Description
MainView	This class centralizes the creation and the management of the simulator's main HMI window. It creates the HMI sub-assemblies: the control panel, the neural network view and the plotting area for the 2D graphics.
ControlPanel	This class allows the creation and the management of the control panel containing the buttons that the user can interact with.
PlotPanel	This class allows the creation and the management of the 2D graphic areas for displaying the simulation's output data over the time.
NetworkView	This class allows the creation and management of the graphic area on which the neural network is displayed (and can interact with user).

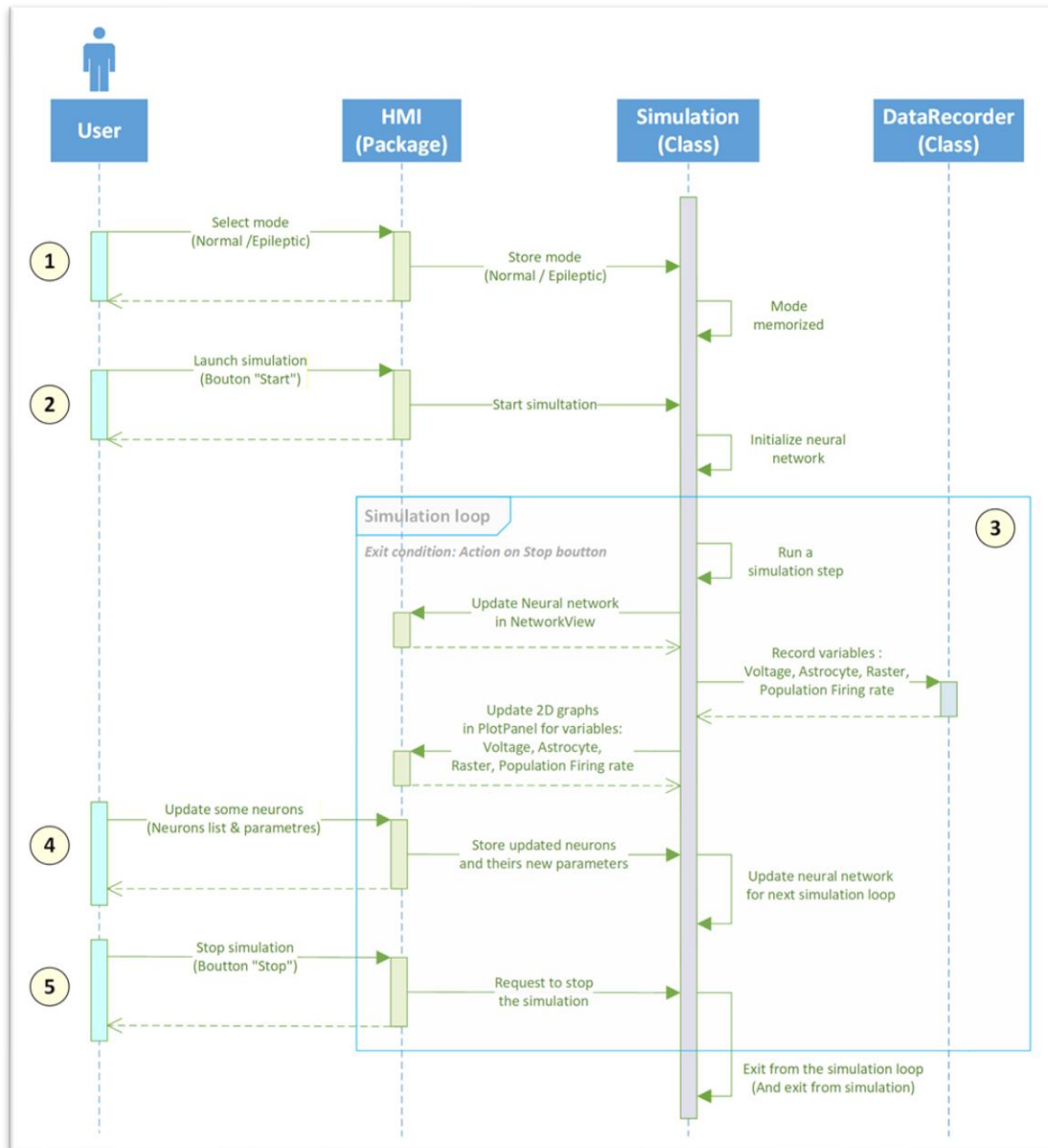
The following diagram illustrates the VIEW package with its classes (attributes and methods):



3.5 Dynamic view

The dynamic view allows to represent the processes flow between the different classes, regardless of the static organization chosen. This dynamic view can include as many sequence diagrams as needed. Here, only the sequence diagram representing the simulation loop is shown as it is the most central one in the simulation software.

The following diagram illustrates the simulation sequence, taking into account the actions requested by the user via the software's HMI:



The elements involved in this sequence are (from left to right):

- The **User** represents the HMI user dealing with the simulator software,
- The **HMI** package includes all the classes within the package. The decision was made to represent only the package itself, rather than the four individual classes, to simplify the diagram. The use of the different classes will be detailed in the next sequence table (see table below).
- The **Simulation** class that performs all the algorithmic treatments,
- The **DataRecorder** class, which is responsible for storing results over time and also allows them to be extracted to a file (This action is not shown in this sequence diagram).

SNN Network Simulation - Documentation

Once the software is already running and awaiting user actions, the simulation loop includes the following sequences:

Sequence	Description
1	The user defines whether to activate « Normal » or « Epilepsy » mode to be taken into account in the simulation algorithm. To do this, user clicks on the corresponding button to choose the selected mode; the ControlPanel class of the HMI package receives this setting and stores the state of this mode in an attribute of the Simulation class.
2	The user starts the simulation by clicking on « Start » button, which is managed by the ControlPanel class of the HMI package. This class activates a method of the Simulation class to initialize all the necessary elements prior launching the first simulation loop
3	This sequence represents the main simulation loop; it is triggered immediately after sequence N° 2. At each simulation loop, some processes are systematically performed, while others depend on user's actions which can be triggered during the simulation, such as sequences N° 4 and N° 5, which are described below. The actions systematically triggered in this sequence are treatments of the Simulation class and are the following: - Launch a simulation step using the simulation parameters and the state of the neural network at T time, - At the end of the simulation step, the new state of the neural network model is available and the NetworkView class is called upon to display this state. - The data from the simulation step (voltage, Astrocyte, Raster, Population firing rate) are stored in the DataRecorder class in order to be used later (In particular, to be exported to a file after a simulation). - This same data is displayed in the 2D graphs of the user interface via the PlotPanel class. Sequences N° 4 and N° 5 are part of sequence N° 3, but as they are conditional on user actions, it was decided to describe them separately.
4	The user decides to modify the characteristics of the neural network. To do this, he will interact with the NetworkView class and modify some neuron's characteristics. The NetworkView class calls the Simulation class, so the user's modifications will be reflected in the neural model resulting from the last simulation step.
5	At any time, the user can stop the simulation by clicking on the « Stop » button in the ControlPanel class of the HMI package. This class allows the simulation loop termination condition to be reached in the Simulation class, thus this stop the simulation.

4 Conclusions

The conclusions after the completion of this project are presented hereafter.

On the neurosciences part:

- Although the network remains small, with only 100 neurons and 10 astrocytes, the differences between normal and epileptic-like activity are subtle and not always immediately visible. Even at this scale, the stochastic nature of the network makes it difficult to reliably distinguish synchronization patterns between the two modes.
- One of the main observed differences is about synchronization:
 - In the normal configuration, neuronal activity is more progressive and distributed across the network. Typically, one neuron spikes, then another, sometimes small groups of two or three neurons, but the activity remains relatively irregular and not globally synchronized.
 - In contrast, the epileptic configuration shows much stronger synchronization. Neurons tend to spike together in recurrent collective bursts. The activity often appears organized into several large synchronized phases that repeat over time, producing more rhythmic and coordinated population activity.
- Important observation concerns the stability of network activity:
 - In the normal configuration, a brief current injection is often sufficient to maintain activity in the network for a relatively long period of time. The activity can continue to circulate through the recurrent connections even after the stimulation stops.
 - The epileptic configuration behaves differently. Sustained activity is more difficult to maintain in a stable way. The network more easily transitions either toward highly synchronized bursts or toward periods where almost nothing happens. In other words, the epileptic network appears more unstable, with activity that is either excessively synchronized or rapidly disappearing instead of remaining in a balanced ongoing state.

On the implementation of the simulation software developed:

- This project allowed me to progress in the following areas: Python programming, development of complex algorithms, using external libraries (Brian2, pyqtgraph, numpy), HMI programming and object-oriented design.
- However, there are two difficulties that I had not properly assessed:
 - The fact that the software is « real-time » implies that the results of a simulation are displayed on the user interface at each calculation step, while the user can interact via this interface by modifying the parameters used in the same simulation. This problem is difficult to address, and I ultimately limited the management of these interactions to the minimum in order to prioritize the development of the simulation algorithm, which is the central focus of this work.
 - Although I had carefully considered and defined the design of this software before starting to code, I often made significant adjustments to the design as I developed it. To improve this point, I see two axes: firstly, to deepen my understanding of object-oriented design concepts, and secondly, to dedicate more time at the beginning of a software project to define more thoroughly the software design before starting the coding phase.
- I will most likely have other developments of this type to carry out during my studies, so it is necessary that I focus on improving these points.

5 Appendix

5.1 Variables mapping

The following table provides a mapping between mathematical variables or expressions and programmatic variables:

Physical variable	Description	Variable used in algorithm
C_m	Membrane capacitance.	Cm
I_{ext}	External injected current.	Iext
I_{Na}	Sodium ionic current.	I _{na}
I_K	Potassium ionic current.	I _k
I_L	Leakage ionic current.	I _l
I_{syn}	Total synaptic current received by a neuron.	I _{syn}
g_{Na}	Maximum sodium conductance.	g _{Na}
g_K	Maximum potassium conductance.	g _K
g_L	Leakage conductance.	g _L
E_{Na}	Sodium reversal potential.	E _{Na}
E_K	Potassium reversal potential.	E _K
E_L	Leakage reversal potential.	E _l
m	Sodium activation gating variable.	m
h	Sodium inactivation gating variable.	h
n	Potassium activation gating variable.	n
α_m	Sodium activation transition rate.	alpha _m
β_m	Sodium deactivation transition rate.	beta _m
α_h	Sodium inactivation transition rate.	alpha _h
β_h	Sodium recovery transition rate	beta _h
α_n	Potassium activation transition rate.	alpha _n
β_n	Potassium deactivation transition rate.	beta _n
g_{ij}	Synaptic conductance between neuron i and j.	g
w_{ij}	Synaptic weight amplitude.	w
τ_{ij}	Synaptic decay time constant.	tau _{syn}
E_{syn}	Synaptic reversal potential.	E _{syn}
A_k	Astrocyte activity variable.	A
τ_A	Astrocyte integration time constant.	tau _A

SNN Network Simulation - Documentation

Physical variable	Description	Variable used in algorithm
$I_k^{coupling}$	Total synaptic activity integrated by astrocyte k .	coupling_input
α	Astrocyte modulation coefficient of synaptic weight.	alpha
β	Astrocyte modulation coefficient of synaptic.	beta
w_0	Baseline synaptic weight.	w0
τ_0	Baseline synaptic decay constant.	tau0
$W \in \mathbb{R}^{N \times N}$	Network connectivity matrix.	Synapses.connect()
N	Number of neurons in the network.	N_NEURONS
N_A	Number of astrocytes in the network.	N_ASTRO
$P_{connect}$	Probability of synaptic connection.	CONNECTION_PROB
$I_{syn,j} = \sum_i g_{ij} (E_{syn} - V_j)$	Total synaptic current accumulated from all incoming synapses.	Compute_synaptic_current()
$\frac{dg_{ij}}{dt} = -\frac{g_{ij}}{\tau_{ij}}$	Synaptic conductance exponential decay.	dg/dt = -g / tau_syn
$\frac{dA_k}{dt} = \frac{-A_k + I_k^{coupling}}{\tau_A}$	Astrocyte temporal integration dynamics.	dA/dt = (-A + coupling_input) / tau_A
$w_{ij} = w_0(1 + \alpha A_k)$	Astrocyte-dependent synaptic potentiation.	w = w0 * (1 + alpha * astro)
$\tau_{ij} = \tau_0 + \beta A_k$	Astrocyte-dependent synaptic modulation.	tau_syn = tau0 + beta * astro

5.2 Python libraries

The following table provides the list of Python binaries libraries (also available in the requirement.txt file) used in the simulation software running under Python 3.12.3:

Library	Release	Library	Release	Library	Release
annotated-types	0.7.0	MarkupSafe	3.0.3	PyQt5	5.15.11
blosc2	4.2.0	matplotlib	3.10.9	PyQt5-Qt5	5.15.2
Brian2	2.10.1	mpmath	1.3.0	PyQt5_sip	12.18.0
brian2tools	0.3	msgpack	1.1.2	pyqtgraph	0.14.0
certifi	2026.4.22	natsort	8.4.0	python-dateutil	2.9.0.post0
charset-normalizer	3.4.7	ndindex	1.10.1	requests	2.34.2
colorama	0.4.6	networkx	3.6.1	setuptools	82.0.1
contourpy	1.3.3	numexpr	2.14.1	setuptools-scm	10.0.5
cycler	0.12.1	numpy	2.4.5	six	1.17.0
Cython	3.1.3	packaging	26.2	sympy	1.14.0
fonttools	4.63.0	pillow	12.2.0	tables	3.11.1
idna	3.15	pip	24.0	threadpoolctl	3.6.0
Jinja2	3.1.6	py-cpuinfo	9.0.0	typing_extensions	4.15.0
kiwisolver	1.5.0	pydantic	2.13.4	typing-inspection	0.4.2

Library	Release
libNeuroML	0.6.7
lxml	6.1.0
markdown-strings	3.4.0

Library	Release
pydantic_core	2.46.4
PyLEMS	0.6.9
pyparsing	3.3.2

Library	Release
urllib3	2.7.0
vcs-versioning	1.1.1

5.3 Installation guide

The prerequisite prior installing the software is to have a Python 3.12.3 release already installed.

To install the simulation software, first you need to create a Python virtual environment and activate it:

<https://python.land/virtual-environments/virtualenv>

Secondly, you need to install dependencies libraries in your environment by running:

```
pip install numpy
pip install pyqt5
pip install pyqtgraph
pip install brian2
pip install brian2tools
```

After this, you are ready to execute the code. Within your activated virtual environment you can go at the root folder of the project (use **cd** and **pwd** commands):

```
.\Projet_SNN
```

And execute the file **main.py** with this command:

```
python main.py
```

In case something goes wrong, use the **requirement.txt** file located in the same folder than the **main.py** file:

```
.\Projet_SNN
```

And execute the following command:

```
pip install --force-reinstall --no-cache-dir -r requirements.txt
```

After doing this, you maybe need to reinstall the 5 libraries above, and retry to run **main.py**.

5.4 User guide

Once you have executed the **main.py** file, a window will be displayed.

At this level, you can click on buttons on the Control Panel located on the left. First, select the mode in which you would like to run the simulation: use « Epileptic » or « Normal » button. The default mode is normal. Once you have selected the mode, you can start the network modelling simulation by using the **Start** button.

When the neural network simulation is running you can inject current into each neuron by clicking on it. When you click on a neuron, this will update the view at the right (Plot Panel) by showing the voltage of the selected neuron. For astrocytes it's the same, but there is no action on the network, it just changes the view. In this view, only historical data are displayed for the selected element (neuron/astrocyte).

For your information, the size of the elements (neuron/astrocyte) is indexed on their voltage and activity.

At any moment, you can click on **pause** or **reset** (these two actions are slow), which suspend or stop the simulation. To change the mode, you need to activate **reset**, switch to the new mode and click on **start** button to start a new simulation. When a simulation is suspended then click on the **start** button to resume it.

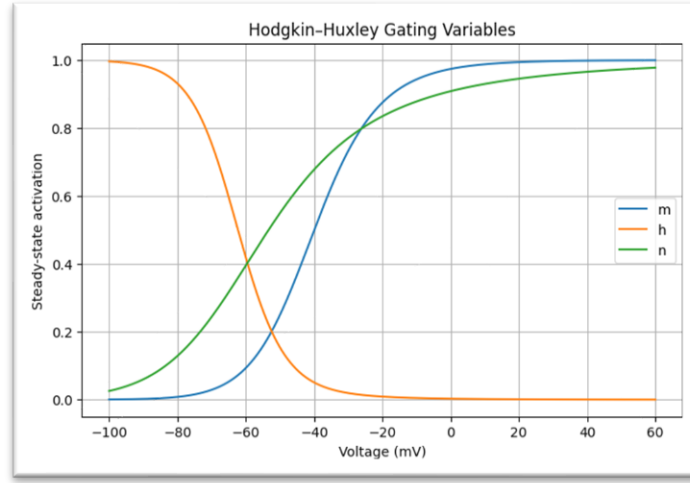
You can export data from a simulation that is completed. This means that you are not allowed to export data during a running simulation, you first need to stop it and then save data in a file by using the « Export Data » button.

Clicking on « Export Data » button, open a new window where you can browse folders and select the file you want to save simulation data.

5.5 Hodgkin-Huxley variables

The goal of this paragraph is to keep track of the equations of Hodgkin-Huxley variables used in modelling; The m, h and n variables are described in the following paragraphs.

The graphic hereafter is presented just to remind the Hodgkin-Huxley representation:



5.5.1 Sodium activation gate (m)

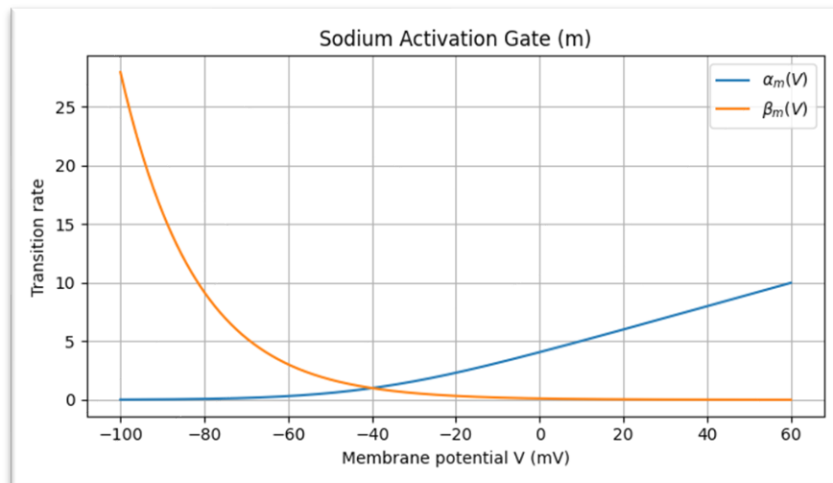
This variable is defined by the following equations and representation:

$$\frac{dm}{dt} = \alpha_m(1 - m) - \beta_m m$$

With:

$$\alpha_m(V) = \frac{0.1(V + 40)}{1 - e^{-(V+40)/10}}$$

$$\beta_m(V) = 4e^{-(V+65)/18}$$



5.5.2 Sodium inactivation gate (h)

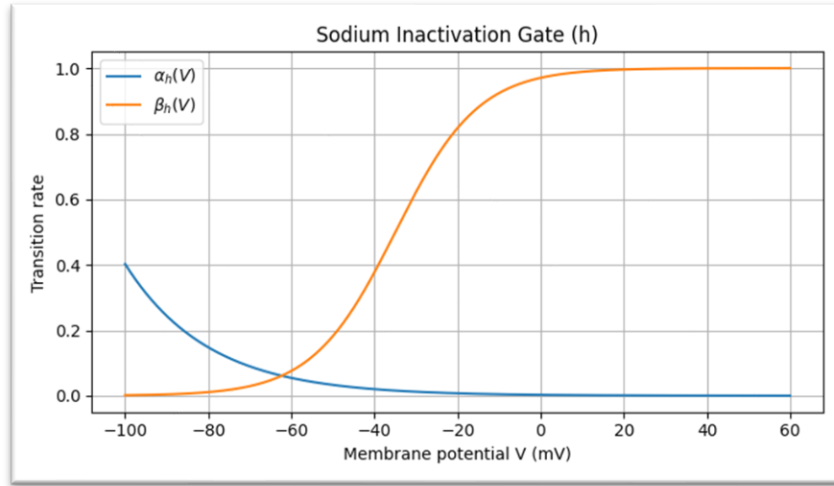
This variable is defined by the following equations and representation:

$$\frac{dh}{dt} = \alpha_h(1 - h) - \beta_h h$$

With:

$$\alpha_h(V) = 0.07e^{-(V+65)/20}$$

$$\beta_h(V) = \frac{1}{1 + e^{-(V+35)/10}}$$



5.5.3 Potassium activation gate (n)

This variable is defined by the following equations and representation:

$$\frac{dn}{dt} = \alpha_n(1 - n) - \beta_n n$$

With:

$$\alpha_n(V) = \frac{0.01(V + 55)}{1 - e^{-(V+55)/10}}$$

$$\beta_n(V) = 0.125e^{-(V+65)/80}$$

